

Poslik Caisse

Mini caisse Android (Kotlin, Jetpack Compose) qui encaisse, imprime et garde son historique avec ou sans réseau, puis se synchronise avec Firebase Realtime Database sans perte ni doublon.

Lancer le projet et les tests

1. Ouvrir le dossier dans Android Studio (JDK 17, SDK 35), placer `google-services.json` dans `app/`, lancer sur une tablette ou un émulateur (Android 8+).
2. Au premier lancement, créer le compte de l'établissement (email et mot de passe) ou s'y connecter, puis saisir un code caisse (C01), qui est réservé dans Firebase. Ces deux étapes demandent le réseau, une seule fois : la session reste ouverte hors ligne.
3. Tests : `./gradlew test` (domaine, Room via Robolectric, ViewModel) et `./gradlew connectedAndroidTest` (UI Compose). La CI joue aussi un scénario de bout en bout sur émulateur Android contre la Firebase Emulator Suite : ventes en ligne, hors ligne, reconnexion sans doublon, panne imprimante, puis redémarrage et réimpression.
4. Firebase : `npx -y firebase-tools@latest deploy --only auth,database` active l'authentification email/mot de passe et publie les règles (`firebase.json`, `firebase/database.rules.json`).

Le menu « Panne imprimante (démon) » simule une imprimante en échec. Sans `google-services.json`, l'app saute la connexion et fonctionne entièrement hors ligne.

Architecture

Trois modules, MVVM et Clean Architecture : `:domain` (Kotlin pur : modèles, cas d'usage, file d'impression, logique de synchro, testés sur la JVM), `:data` (Room, Firebase, WorkManager, imprimante simulée) et `:app` (Compose, ViewModels, Hilt). L'authentification passe par Firebase Auth (email/mot de passe), derrière une interface du domaine. **Room est la seule source de vérité** : l'écran, l'impression et la synchro lisent la base locale, et Firebase n'en est qu'une copie.

Schéma de données

- Room : `register_config` (code caisse, dernier numéro) ; `sales` (`sale_id` UUID, `register_code`, `ticket_number`, `total_millimes`, `created_at`, `print_status`, `version`, `synced_version`, `sync_conflict`), avec un index unique sur (`register_code`, `ticket_number`) ; `sale_lines` (produit, prix et quantité copiés au moment de la vente). Les montants sont en millimes (Long), sans arrondi.
- Firebase : `/registers/{code}` (`claimedBy` = uid du compte, `deviceId` = identifiant de l'installation) et `/sales/{code}/{numéro sur 6 chiffres}` (la vente complète).

Unicité des numéros de ticket

Un numéro est code caisse + compteur de la caisse (C01-000042). Le compteur est lu, incrémenté et la vente insérée **dans la même transaction SQLite** : deux appuis simultanés ne peuvent pas obtenir le même numéro, et un échec annule tout, sans trou. Le code caisse est réservé par transaction Firebase au premier lancement, au nom du compte **et** de l'installation : deux tablettes, même connectées au même compte, n'ont jamais le même préfixe. Deux tablettes qui vendent hors ligne en même temps continuent chacune leur série (C01-..., C02-...), sans collision possible.

Pourquoi pas un compteur global : une transaction Firebase exige le serveur, ce qui casse l'encaissement hors ligne ; un numéro provisoire renuméroté à la synchro changerait un ticket déjà imprimé.

Limites. Il n'existe pas de séquence unique tous postes confondus (l'historique global se trie par date, et l'horloge d'une tablette peut dériver). Le premier lancement doit être en ligne. Une réinstallation crée une nouvelle identité : la tablette doit prendre un nouveau code. Si deux tablettes partageaient malgré tout un code (clonage), les règles Firebase refusent d'écraser un ticket portant un autre `saleId` : la vente est marquée « Conflit de numéro », jamais écrasée. Une caisse reste liée au compte qui l'a réservée ; la déconnexion est refusée hors ligne ou tant que des ventes attendent la synchro, pour ne bloquer ni la caisse ni l'envoi.

Non-perte des tickets

Encaisser écrit d'abord la vente dans Room (durable), puis rend la main ; l'impression et la synchro suivent en arrière-plan. La file d'impression traite un ticket à la fois et écrit `PRINTED` ou `FAILED`. Au démarrage, les tickets `PENDING` et `FAILED` repartent à l'impression ; un ticket `PRINTED` n'y repart jamais. Si l'app est tuée entre l'impression physique et l'écriture de l'état, le ticket est réimprimé : on préfère un doublon papier à un ticket manquant.

Hors ligne et reconnexion

Sans réseau, tout fonctionne : numéro, vente, impression, historique. Chaque modification incrémente `version`. Un `SyncWorker` (WorkManager, contrainte réseau, backoff exponentiel, survit au redémarrage) envoie les ventes où `synced_version < version` et ne les marque qu'après l'acquittement du serveur, et seulement si la version n'a pas bougé entre-temps. La clé Firebase est le numéro de ticket et l'écriture remplace le nœud entier : une relance réécrit le même nœud, **jamais de doublon**. Un bandeau indique au caissier l'état du réseau et le nombre de ventes en attente.